

## 20. Řízení přístupu do kritické sekce pomocí SW a HW metod, SW algoritmy, výchozí předpoklady pro HW řešení, algoritmy využívající HW, nedostatky HW i SW metod.

### SW algoritmy

- Petersonův algoritmus
  - řešení pouze pro 2 procesy !!
  - (signalizace připravenosti procesu  $P_i$  ke vstupu do KS nastavením `flag[i] = true` a obdržení pokynu ke vstupu `turn == i`)
- Bakery algoritmus
  - řešení pro n-procesů
  - před vstupem do KS dostane každý proces své číslo. Do KS může vstoupit jen držitel nejmenšího čísla. Dostanou-li  $P_i$  a  $P_j$  stejné, pak přednost má  $P_{\min(i, j)}$

### Výchozí předpoklady pro HW řešení

- Využití speciálních instrukcí procesoru
- proces je vykonáván procesorem kontinuálně, dokud nevyvolá službu OS (systémové volání) nebo není přerušen přerušovacím signálem (např. časovače)
- k přerušení procesu může dojít pouze na hranicích instrukcí
  - po dokončení jedné instrukce a před zahájením další instrukce
- přístup k operační paměti je obvykle výlučný
  - důležité zejména budeme-li uvažovat také víceprocesorové systémy

### Algoritmy využívající HW

#### Zákaz přerušování

```
repeat
    cli;                // instrukce pro zákaz obsluhy přerušování
    // kód KS
    sti;                // instrukce povolení obsluhy přerušování
    // kód ZS
forever
```

Využívá se instrukce zákazu přerušování, kdy daný proces nemůže být přerušován ničím jiným - pouze OS při systémovém volání. Pokud tedy není v KS systémové volání - je vzájemné vylučování splněno.

#### Nevýhody:

- nefunkčnost na víceprocesorových systémech
  - proces z jiného procesoru může do KS vstoupit
- instrukce zákazu přerušování jsou povolené pouze v privilegovaném režimu
- zvyšování latence systému (doba mezi okamžikem příchodu požadavku na přerušování a

okamžikem, kdy se začne provádět odpovídající obslužný program

### Návrh instrukce test\_and\_set

Funkce test\_and\_set zaručí atomicitu operace čtení a zápisu.

```
locked = 0;           // sdílená proměnná evidující obsazenost KS

// Kód procesů:
repeat
    while (!test_and_set(&locked)); // čekání, dokud není KS volná
    // kód KS
    locked = 0; // uvolnění KS
    // kód ZS
forever
```

Možnost použití na SMP systémech.

### Instrukce xchg

Architektura Intel x86 má instrukci xchg, která atomicky vymění obsah dvou operandů (dvou registrů nebo registru s proměnnou v paměti). Lze ji tedy využít jako instrukci test\_and\_set.

```
locked = 0; // sdílená proměnná (v paměti) evidující obsazenost KS

// Kód procesů:
repeat
    test = 1;           // lokální proměnná (v registru) v každém
                        // procesu
    while (test)         // čekání, dokud je KS obsazená
        xchg test, locked; // atomicky přečte a nastaví locked
    // kód KS
    locked = 0;          // uvolnění KS
    // kód ZS
forever
```

### **Nedostatky HW a SW metod**

Nedostatky SW:

- aktivní čekání čekajících procesů zbytečně plýtvá časem procesoru
  - nevhodné pro dlouhé KS
  - vhodné pro velmi krátké KS
  - efektivnější je tedy čekající procesy blokovat (nutná asistence OS)
- chyba, která nastane v KS způsobí vyhladovění ostatních procesů čekajících na KS, protože proces, který v KS způsobil chybu, se stále jeví, jako že v KS stále je
- pokud proces s nižší prioritou nestačil opustit KS a do KS požaduje vstup proces s vyšší

prioritou, vznikne deadlock!

Nedostatky HW:

- neodstraňují nedostatky SW řešení
- stále je potřeba aktivní čekání
- řeší pouze vzájemné vylučování
  - ostatní musí být ošetřeno algoritmicky